
IA NO DESENVOLVIMENTO DE SOFTWARE

Agentes de IA no nosso dia a dia

Como o Claude Code e o superpowers mudam a forma de programar

Rafael Dias · Desenvolvedor Backend · Squad Backoffice & Importação · iClinic / Afya

Primeiro: o que é um LLM?

- ▶ LLM = 'Large Language Model' (modelo de linguagem) — o motor por trás do ChatGPT e do Claude
- ▶ Foi treinado lendo uma quantidade gigantesca de texto: código, documentação, livros, a internet
- ▶ Na essência ele faz UMA coisa: dado um texto, prevê a continuação mais provável
 - É um 'autocompletar' turbinado — não tem uma tabela de verdades absolutas guardada
- ▶ Por isso às vezes erra com confiança (a famosa 'alucinação') → verificar é essencial
 - Analogia: um colega que leu quase tudo e responde na hora — brilhante, mas você confere antes de confiar

O que é uma Skill?

- ▶ Um pacote de instruções que ensina o agente a fazer UMA tarefa bem
 - Na prática: um arquivo de texto (SKILL.md) com um nome, um 'quando usar' e um passo a passo
- ▶ 'Prompt solto' = a instrução que você digita na hora e some quando a conversa acaba
- ▶ A skill é o oposto: o manual escrito UMA vez — versionado, reaproveitável e igual pro time todo
- ▶ Sempre consistente: o agente segue o jeito certo combinado, não o improvisado do dia

superpowers: o pipeline

- ▶ 'Workflow determinístico, com fases travadas, estado persistente e auditável'
- ▶ 5 fases: design → plan → implement → review → complete
 - O estado (design, plano) vive em disco: dá pra parar hoje e retomar semana que vem
- ▶ Você é quem libera cada fase; antes de passar, uma checagem automática confirma que a etapa terminou de verdade
 - O agente nunca pula fase sozinho — nem 'isso é simples demais pra ter design'
 - O time já rodou dezenas de features reais com esse fluxo

ai-jail: o que é (a cerca de segurança do agente)

- ▶ ai-jail é um 'cercado de segurança': um espaço isolado onde o agente roda preso, sem acesso ao resto da máquina
- ▶ Analogia: é como o cercadinho da criança — ela brinca à vontade lá dentro, mas não sai pra rua
- ▶ Exemplo: você quer deixar o Claude Code refatorar sozinho. Sem ai-jail, ele vê TUDO (arquivos, senhas do .env, internet)
 - Com ai-jail você diz: 'só esta pasta, sem .env, sem internet' — e ele trabalha preso nesse cercado
- ▶ Feito pelo Fabio Akita, roda no terminal e funciona com Claude Code, Cursor e outros
 - Ou seja: ele NÃO escreve código — é a cerca que fica EM VOLTA do agente

MÃO NA MASSA

Hands-on: vamos ao VS Code

Três formas ao vivo: Claude Code “cru” → superpowers (processo) → ai-jail (sandbox).

Boas práticas de mercado (2026)

Recomendado

- Peça um PLANO antes de deixar o agente codar (o design vem primeiro)
- Trabalhe em passos pequenos e verificáveis, com testes rodando
- Reveja o diff sempre — a responsabilidade pelo código é sua
- Padronize o time com skills versionadas no repositório
- Isole execuções autônomas em sandbox, ainda mais com dados sensíveis

Evite

- 'Vibe coding': aceitar o que foi gerado sem ler
- Dar acesso total/autônomo sem sandbox e sem backup
- Colar segredos (tokens, .env) no prompt ou no contexto
- Confiar no 'está pronto' sem evidência (testes verdes)
- Instalar skills/plugins de terceiros sem revisar antes

Próximos passos para o time

- ▶ Padronizar nossas skills de projeto no repo (test-runner, convenções, review)
- ▶ Usar o superpowers como fluxo padrão para features de ponta a ponta
- ▶ Definir política de segurança: sandbox para execuções autônomas + dados sensíveis
- ▶ Formar o time: do 'prompt solto' para 'skills + verificação'
- ▶ Medir: tempo de ciclo, retrabalho e cobertura antes/depois

OBRIGADO

Obrigado a todos pela atenção

Repositórios dos frameworks

- superpowers
github.com/obra/superpowers
- superpowers (obra / Jesse Vincent)
github.com/obra/superpowers
- ai-jail (Fabio Akita)
github.com/akitaonrails/ai-jail

Para ir além

- Claude Code — documentação oficial
docs.claude.com/en/docs/claude-code
- superpowers — post do autor
blog.fsck.com/2025/10/09/superpowers
- ai-jail — artigo do Akita
akitaonrails.com